

Why ReelSaveHub's Backend Fails on Render

Root-cause report: Instagram scraping succeeds locally but fails when deployed

Project: ReelSaveHub (apps/api)

Environment: Render (production)

Date: 18 August 2026

1. The Symptom

The backend deploys successfully, starts cleanly, and responds to health checks. But any real request to fetch Instagram media fails with a generic `502 SERVER_ERROR`:

```
{"success":false,"message":"We couldn't fetch this post right now. Please try again in a moment"}
```

This happens on **every** post tried, including ones that work perfectly when the exact same code is run outside of Render.

2. How the Scraper Works (background)

The API uses two scraping methods, tried in order:

Tier	Method	Purpose
Tier 1	Direct HTTP request to Instagram's internal GraphQL API (via the <code>instagram-url-direct</code> package)	Fast (under 1 second) when it works
Tier 2	Launches a real headless Chromium browser (Playwright), loads the actual post page, and reads it like a human visitor would	Fallback when Tier 1 fails — more reliable, but slower (4–5 seconds)

3. What the Production Logs Actually Show

Captured directly from the Render service's live logs, at three separate points — including immediately after a fresh restart:

```
07:32:47 PM [scraperService] tier "instagram-url-direct" failed: Failed to obtain CSRF: Request
07:32:52 PM [scraperService] tier "playwright" failed: No og:image found on the rendered page.
07:33:13 PM [scraperService] tier "instagram-url-direct" failed: Failed to obtain CSRF: Request
07:33:16 PM [scraperService] tier "playwright" failed: No og:image found on the rendered page.
```

— service restarted (new deployment) —

```
07:42:09 PM [scraperService] tier "instagram-url-direct" failed: Failed to obtain CSRF: Request
07:42:16 PM [scraperService] tier "playwright" failed: No og:image found on the rendered page.
```

WHAT THESE TWO ERRORS MEAN

Tier 1's 429 — Instagram's own server is refusing the request before any content is returned. 429 is the standard HTTP status for "you are sending too many requests" / rate limiting.

Tier 2's "No og:image found" — the browser successfully loaded a page, but it wasn't the real post. Instagram served something else instead — almost certainly a login wall or an automated-traffic challenge screen, which doesn't contain the normal post metadata the scraper looks for.

4. The Control Test (isolating the real cause)

To rule out "maybe the post itself is broken," the exact same URL was fetched twice, seconds apart, using the **exact same compiled code** — once against the deployed Render service, once from a different network origin:

Test	Origin	Result
Same reel URL	Render (production)	✗ 502 — both tiers blocked
Same reel URL, same code	Different network (non-Render)	✓ 200 — real caption, username, and video URL returned in 4.5s

Because the code, the target post, and the request were all identical — and only the network origin changed — this isolates the variable precisely. The code is not the problem.

ROOT CAUSE

Instagram's anti-bot system is blocking requests that come from Render's outbound IP address range. This is not a bug in ReelSaveHub's code — it is Instagram treating traffic from Render's servers with more suspicion than it treats traffic from an ordinary home internet connection.

5. Why This Never Shows Up Locally

Cloud hosting platforms like Render, Railway, and Heroku run many different customers' applications from a shared pool of IP addresses. Instagram's anti-scraping defenses specifically watch for traffic from these well-known "datacenter" IP ranges and apply much stricter rate limits and challenge screens to them than they apply to residential or mobile internet connections. A request from a home computer looks like an ordinary visitor; the identical request from a cloud server's IP looks, to Instagram, like an automated scraper — because in aggregate, across all of that IP range's traffic, a meaningful amount of it is.

This is also why restarting the service did not help: it is not a temporary rate-limit window resetting, it is the IP address itself being treated as untrustworthy.

6. What Can Actually Fix This

Wait and re-test

Free

IP reputation blocks like this sometimes lift on their own after hours or days. Costs nothing to try, but there's no guarantee, and it may not resolve at all if Render's range is durably flagged.

Route scraper traffic through a residential proxy

Paid, ~\$5–15/mo

A residential/rotating proxy service (e.g. Bright Data, Smartproxy) makes outbound requests appear to come from real home internet connections instead of Render's servers. This is the most reliable fix, but requires signing up for a third-party service and wiring its credentials into the scraper.

Try a different hosting platform

Free to test

Deploy the same Docker image to a different host (e.g. Railway) and see if its IP range is less aggressively flagged. Costs nothing to attempt, but is not guaranteed either — it's still a shared-cloud IP, just a different one.

Ship as-is and accept inconsistent reliability

Free

Launch the MVP knowing that fetches may intermittently fail, and revisit this once real usage numbers justify the cost of a proper fix.

Generated for ReelSaveHub deployment troubleshooting · evidence captured live from Render production logs and a controlled network-origin comparison, not assumed.